Empório da Música — Agente de Atendimento com IA & RAG

Projeto desenvolvido para o Desafio Técnico de AI Engineer da Artefact. Trata-se de um assistente virtual conversacional para a loja física e e-commerce da Empório da Música (Campo Grande/MS), combinando arquitetura ReAct (Reasoning + Acting), Model Context Protocol (FastMCP), recuperação semântica sobre políticas (RAG), memória persistente em SQLite, resiliência multi-provedor (Groq, Gemini, OpenAI) e uma interface frontend interativa desenvolvida em Next.js com o mascote Téo.

Quickstart (Execução Rápida)

```
# 1. Configurar ambiente virtual e instalar dependências
python -m venv venv && source venv/bin/activate  # Windows:
.\venv\Scripts\activate
pip install -r requirements.txt && cp .env.example .env

# 2. Executar ETL (SQLite) e subir o servidor FastAPI com hot-reload
python main.py --etl
python main.py --server  # Acessível em http://localhost:8000 (Docs: /docs)
```

Sumário

1. Visão Geral e Persona do Agente
2. Arquitetura da Solução e Diagrama de Fluxo
3. Decisões Técnicas e Tratamento de Regras de Negócio
4. Instruções de Instalação e Execução
5. Suíte de Avaliação Automatizada (Evals) e Testes Unitários
6. Exemplos de Atendimento e Casos Não Triviais
7. Limitações Conhecidas e Próximos Passos (Com Mais Tempo)
8. Relato de Workflow e Uso de Assistentes de IA

---

1. Visão Geral e Persona do Agente

A Empório da Música é uma loja tradicional localizada na Rua 14 de Maio, 3200 — Centro, Campo Grande/MS. O objetivo do projeto foi desafogar a equipe de atendimento humano, automatizando o suporte a dúvidas recorrentes (horários, endereço, disponibilidade de produtos, preços, cálculo de parcelas e status de pedidos), sem perder o acolhimento e a autoridade técnica.

Persona: Téo

Para o agente, estruturei a persona do **Téo**, o especialista musical virtual da loja:

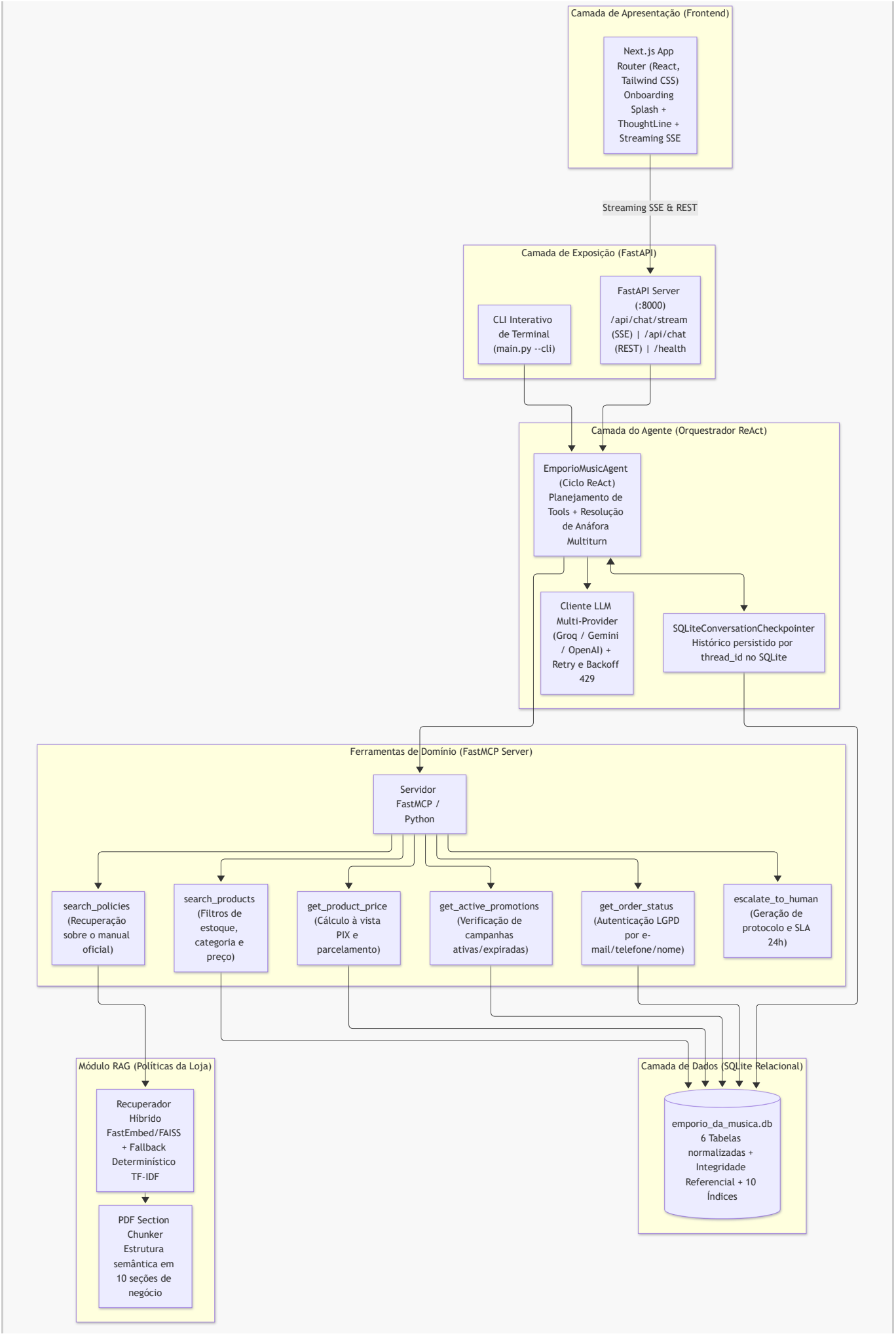
- **Tom de voz:** Cordial, consultivo, paciente e próximo da realidade dos músicos (desde iniciantes com orçamento restrito até instrumentistas experientes).

- **Escopo restrito:** A loja comercializa exclusivamente instrumentos musicais completos. A persona não vende e não alucina preços de acessórios avulsos (encordoamentos, palhetas, cabos, cases ou pedais), orientando educadamente o cliente sobre o escopo exclusivo da loja física em instrumentos musicais.
- **Rigor com dados e privacidade:** O Téo não expõe dados de pedidos de terceiros (conformidade com LGPD) e nunca inventa condições comerciais fora das tabelas vigentes do banco.

---

2. Arquitetura da Solução e Diagrama de Fluxo

Optei por construir uma arquitetura desacoplada em camadas, evitando dependências monolíticas pesadas e garantindo controle total sobre o ciclo de execução:


Camada de Apresentação (Frontend)
Next.js App Router (React, Tailwind CSS) Onboarding Splash + ThoughtLine + Streaming SSE
Streaming SSE & REST
Camada de Exposição (FastAPI)
CLI Interativo de Terminal (main.py --cli)
FastAPI Server (:8000) /api/chat/stream (SSE) | /api/chat (REST) | /health
Camada do Agente (Orquestrador ReAct)
EmporioMusicAgent (Ciclo ReAct) Planejamento de Tools + Resolução de Anáfora Multiturn
Cliente LLM Multi-Provider (Groq / Gemini / OpenAI) + Retry e Backoff 429
SQLiteConversationCheckpointer Histórico persistido por thread_id no SQLite
Ferramentas de Domínio (FastMCP Server)
Servidor FastMCP / Python
search_policies (Recuperação sobre o manual oficial)
search_products (Filtros de estoque, categoria e preço)
get_product_price (Cálculo à vista PIX e parcelamento)
get_active_promotions (Verificação de campanhas ativas/expiradas)
get_order_status (Autenticação LGPD por e-mail/telefone/nome)
escalate_to_human (Geração de protocolo e SLA 24h)
Módulo RAG (Políticas da Loja)
Recuperador Híbrido FastEmbed/FAISS + Fallback Determinístico TF-IDF
PDF Section Chunker Estrutura semântica em 10 seções de negócio
Camada de Dados (SQLite Relacional)
emporio_da_musica.db 6 Tabelas normalizadas + Integridade Referencial + 10 Índices

3. Decisões Técnicas e Tratamento de Regras de Negócio

Durante a exploração dos arquivos do desafio (`.csv` e `.pdf`), identifiquei particularidades de negócio que foram tratadas diretamente nas ferramentas para garantir que o agente não alucinasse:

Regra de Negócio / Pegadinha	Desafio Encontrado	Solução Implementada
Takamine GD20 (Promoção Expirada)	A tabela `promotions` possui desconto de 12% para o produto ID 95, mas com `is_active = 0`.	A tool `get_product_price` valida o campo `is_active`, informa ao cliente que a campanha encerrou e apresenta o preço de tabela oficial: **R$ 2.199,00** (ou **R$ 2.089,05** no PIX com 5% à vista).
Violões até R$ 1.000 (Itens Indisponíveis)	O catálogo contém o *Giannini GF-3D* com estoque zerado e o *Shelby SN-7C* com status `discontinued`.	A tool `search_products` aplica por padrão o filtro `only_available = True` (`status = 'active'` e `stock_quantity > 0`), retornando apenas opções reais para compra como *Tagima Memphis AC-39* e *Michael Taurus*.
Não-Cumulatividade do PIX	Risco de aplicar 5% de desconto no PIX sobre itens já com desconto promocional ativo.	O cálculo em código bloqueia a aplicação dos 5% do PIX se o produto possuir promoção ativa (`is_active = 1`), instruindo o cliente com transparência.
Privacidade e LGPD em Pedidos	A consulta aberta de pedidos exporia código de rastreio, endereço e valor da compra a terceiros.	A tool `get_order_status` exige confirmação do identificador do titular (e-mail, telefone ou nome completo). Se o usuário informar apenas `#1`, a exibição é bloqueada e os dados são solicitados. Se um terceiro tentar dados falsos, a consulta é negada sem vazar os dados reais.
Escopo Exclusivo em Instrumentos	O cliente pode solicitar palhetas, encordoamentos avulsos ou cabos, que a loja não comercializa.	Mapeei termos de acessórios e temas gerais (culinária, futebol). A consulta de acessórios aciona a Seção 1 das políticas para embasar a recusa cordial.
Direito de Arrependimento e Frete Reverso	Aplicação das regras do Código de Defesa do Consumidor (CDC) para compras online.	O RAG recupera a Seção 4 das políticas oficiais, informando o prazo legal de 7 dias corridos, exigência de embalagem íntegra e gratuidade total do frete reverso custeado pela loja.
Escalação para Atendimento Humano	Avarias de transporte (ex: instrumento que chegou com braço quebrado).	A tool `escalate_to_human` gera protocolo oficial (`EMP-AAAAMMDD-XXXX`), registra a ocorrência e fornece o prazo oficial de SLA (24h úteis) e o WhatsApp da equipe.
Resolução de Anáfora Multiturn	O cliente pergunta "*Quais guitarras vocês têm até*	O orquestrador salva as saídas estruturadas das tools na memória SQLite e resolve pronomes

Regra de Negócio / Pegadinha	Desafio Encontrado	Solução Implementada
	1500?" e depois pergunta "*E a primeira, tem microfone duplo?*".	("*dela*", "*do primeiro*", "*no pix hoje*") resgatando o modelo exato do histórico recente.

Premissas e Suposições de Negócio Adotadas

Para guiar a implementação onde os dados de entrada eram ambíguos ou abertos a interpretação, adotei as seguintes premissas:

1. **Não-cumulatividade do PIX:** Assumi que o desconto de 5% à vista no PIX incide sobre o preço de tabela e não se acumula com produtos em liquidação ou promoções sazonais ativas (`is_active = 1`), preservando a margem da loja e a coerência comercial.
2. **Validação cruzada de dados cadastrais (LGPD):** Assumi que o número do pedido isolado não confere legitimidade de acesso. Para respeitar o princípio da segurança e prevenção da LGPD (Lei 13.709/2018), exigi a validação cruzada do número do pedido com ao menos um dado confirmador do titular cadastrado (e-mail, telefone ou nome completo).
3. **Fidelidade ao catálogo e estoque real:** As categorias *Instrumentos de Sopro* e *Cordas Orquestrais* constam no CSV de categorias, mas não possuem produtos cadastrados no CSV de produtos. Assumi que o agente deve informar com transparência a falta momentânea desses itens no estoque, sem alucinar marcas ou preços inexistentes.
4. **Resiliência e localização dos dados brutos:** Mantive os arquivos originais em `arquivos/` e construí o pipeline de ETL e o chunker para resolver caminhos automaticamente tanto em `arquivos/` quanto em `data/` (com ou sem o prefixo original), consolidando a base relacional normalizada em `data/emporio_da_musica.db`.

Justificativas de Tecnologias Utilizadas e Trade-offs Arquiteturais

- **Abordagem ReAct Modular Própria vs Frameworks Monolíticos (LangChain / LlamaIndex):**
  - *Trade-off Consciente:* Frameworks como LangChain e LlamaIndex frequentemente adicionam camadas espessas de abstração opaca, sofrem com quebras constantes de retrocompatibilidade em atualizações de versão e introduzem overhead computacional desnecessário.
  - *Decisão:* Optei por implementar o loop ReAct, o gerenciador de memória transacional e a resolução de anáfora diretamente em Python puro com schemas tipados (**Pydantic**). Isso garante **100% de controle sobre o ciclo de raciocínio**, facilidade de depuração, performance máxima e imunidade a quebras de API externas, além de demonstrar domínio dos conceitos fundamentais de agentes em vez de mero uso de abstrações pré-fabricadas.
- **Armazenamento com SQLite Relacional + WAL Mode vs PostgreSQL/Vector DBs Dedicados:**
  - *Trade-off Consciente:* Um banco como PostgreSQL com extensão `pgvector` ou clusters como Pinecone/Qdrant seriam escolhas padrão para produção enterprise com alta concorrência de escrita. No entanto, exigiriam subir múltiplos containers Docker (`docker-compose`), criar volumes persistentes e gerenciar portas de rede na máquina de avaliação.
  - *Decisão:* Para o contexto do desafio, priorizei a **Portabilidade Absoluta (Zero-Friction Evaluation)**. O SQLite entrega execução instantânea em qualquer sistema operacional sem nenhuma dependência de infraestrutura externa. Para garantir qualidade de produção, configurei `PRAGMA foreign_keys = ON`, `PRAGMA journal_mode = WAL` (Write-Ahead Logging para

concorrência de leitura/escrita) e criei **10 índices relacionais** estratégicos nas tabelas de produtos, pedidos, clientes e promoções. Toda a camada de dados foi isolada sob o padrão *Repository*, permitindo migração imediata para PostgreSQL em menos de 1 dia de trabalho.

- **Python como linguagem principal:** Utilizado no backend, orquestração e scripts de avaliação, cumprindo o requisito obrigatório do desafio.
- **FastMCP em vez de dependências pesadas de agentes:** Optei por estruturar as ferramentas sob o padrão Model Context Protocol (FastMCP). Isso mantém as ferramentas desacopladas do modelo, fáceis de testar unitariamente e prontas para integração externa.
- **Arquitetura de Retrieval (RAG):**
  - **Chunking Semântico:** Em vez de usar divisores cegos por quantidade de caracteres (que cortam regras no meio), dividi o arquivo `políticas_da_loja.pdf` estritamente pelas suas **10 seções temáticas de negócio** estruturadas em `data/policies_sections.json`.
  - **Embeddings & Vetorização:** Utilizei o modelo multilíngue `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` gerido via `FastEmbed`, que oferece suporte nativo ao português brasileiro (vetores de 384 dimensões), com indexação vetorial em memória através de **FAISS**, recuperando as `top_k = 3` seções mais relevantes por consulta.
  - **Fallback Determinístico Local:** Caso o ambiente execute sem conectividade para download de pesos, o recuperador aciona automaticamente uma busca determinística baseada em TF-IDF com char n-grams `[3, 4]` e expansão por dicionário de sinônimos musicais.
- **Provedor e Escolha do Modelo de LLM:**
  - **Provedor (Groq):** Homologuei a solução prioritariamente na **Groq** por dois fatores decisivos:
    1. *Velocidade de Inferência:* O processamento via LPUs entrega respostas em streaming com latência de ~1 segundo, proporcionando uma experiência de chat fluida.
    2. *Acessibilidade e Custo Zero:* A Groq disponibiliza um plano gratuito generoso sem exigir cartão de crédito internacional. Isso garante que qualquer avaliador da Artefact consiga clonar o repositório, gerar uma chave gratuita em segundos e rodar o projeto integralmente sem custos de infraestrutura.
  - **Modelo:** Padronizado no **`openai/gpt-oss-120b`**, modelo open-weights de alta capacidade de raciocínio da OpenAI hospedado na Groq (com janela de contexto de 131k tokens, geração com reasoning interno e suporte robusto para o português brasileiro).
- **Estratégia de Prompting e Guardrails:**
  - **System Prompt Estruturado:** Definido em `src/agent/prompts.py`, separando rigidamente a Persona (Téo), os Guardrails Comerciais Estritos e as Diretrizes de Formatação (proibição expressa de tabelas Markdown com pipes, privilegiando tópicos limpos e destaque de valores no PIX).
  - **Augmented Context Injection:** As ferramentas retornam payloads estruturados que são injetados em blocos delimitados (`[Resultado da Tool]`) antes da geração final, forçando o modelo a ancorar toda afirmação numérica nos dados retornados pelas tools.
  - **Heurística contra Prompt Injection e Descontos Indevidos:** O orquestrador detecta tentativas de burlar regras ("amigo do dono", "autorizado pelo gerente") e injeta um alerta no prompt instruindo o modelo a recusar o desconto e manter os valores oficiais. Reconheço que se trata de uma proteção heurística em código voltada aos padrões mapeados; para produção aberta, a recomendação é integrar um classificador de segurança dedicado (como Llama Guard ou NeMo Guardrails).
- **Cliente LLM com Retry e Backoff Exponencial:** Para contornar limites transitórios de taxa (HTTP 429) em APIs públicas, implementei um decorador com backoff exponencial com jitter que tenta até 3 vezes

antes de acionar o sintetizador determinístico de resiliência.
- **Higienização de RAG e Fallback de Contingência:** Caso ocorra indisponibilidade temporária de rede externa com a API de LLM, o orquestrador aciona um sintetizador determinístico grounded. O mecanismo higieniza marcadores técnicos de chunks (`--- [...] ---`), entregando respostas formatadas e acolhedoras no tom da persona Téo sem expor metadados internos.

---

4. Instruções de Instalação e Execução

Pré-requisitos

- Python 3.11 ou superior
- Node.js 18 ou superior (para executar a interface web)
- Chave de API de LLM (recomendo Groq gratuita ou Google Gemini)

1. Clonar o Repositório e Configurar Ambiente

```
git clone <URL_DO_SEU_REPOSITORIO>
cd desafio-artefact

# Criação e ativação do ambiente virtual
python -m venv venv

# No Windows:
.\venv\Scripts\activate
# No Linux/macOS:
source venv/bin/activate

# Instalação das dependências
pip install -r requirements.txt
```

2. Configurar Variáveis de Ambiente (`.env`)

Copie o arquivo de exemplo e insira suas credenciais:

```
cp .env.example .env
```

Exemplo de configuração com Groq (recomendado — modelos open-weights de raciocínio da OpenAI hospedados com inferência ultra-rápida):

```
LLM_PROVIDER=groq
LLM_MODEL=openai/gpt-oss-120b
GROQ_API_KEY=gsk_sua_chave_groq_aqui
DATABASE_PATH=data/emporio_da_musica.db
```

(Também é compatível com o modelo `openai/gpt-oss-20b` caso deseje latência ainda menor). Caso prefira usar o Google Gemini:

```
LLM_PROVIDER=google
LLM_MODEL=gemini-2.5-flash
GOOGLE_API_KEY=sua_chave_gemini_aqui
DATABASE_PATH=data/emporio_da_musica.db
```

3. Executar o Pipeline de ETL (Criação do Banco SQLite)

```
python main.py --etl
```

Este comando lê os 6 arquivos CSV em `arquivos/`, aplica as validações de schema com Pydantic, cria as tabelas com integridade referencial e 10 índices em `data/emporio_da_musica.db`, indexa as políticas e gera o relatório em `docs/data_cleaning.md`.

4. Executar os Testes Unitários

```
python -m unittest discover -s tests -p "test_*.py" -v
```

Executa a suíte completa de **37 testes automatizados** cobrindo ETL, banco relacional, RAG, ferramentas FastMCP, orquestrador ReAct, defesas LGPD e streaming SSE.

5. Executar o Servidor FastAPI (Backend)

```
python main.py --server
```

O servidor iniciará em `http://localhost:8000` com hot-reload ativado para facilitar o desenvolvimento local. A documentação OpenAPI interativa pode ser consultada em `http://localhost:8000/docs`.

6. Executar o Modo CLI (Linha de Comando)

Caso queira testar a conversação do agente diretamente no terminal:

```
python main.py --cli
```

7. Executar a Interface Web (Frontend Next.js)

Em outro terminal:

```
cd frontend
npm install
npm run dev
```

Acesse `http://localhost:3000` no seu navegador para interagir com a interface completa do Téo.

---

5. Suíte de Avaliação Automatizada (Evals) e Testes Unitários

Para mensurar de forma contínua e repetível se o agente está acionando as ferramentas corretas e respeitando os guardrails de negócio, construí um runner de avaliação automatizada (`evals/runner.py`) que consome os cenários declarados em `evals/cases.yaml`.

Cada caso é avaliado deterministicamente através de asserções de contrato em três camadas:

1. **Contrato de Ferramentas:** Verifica se o orquestrador acionou exatamente a(s) tool(s) correta(s) para a intenção.
2. **Termos Obrigatórios (*Must-have*):** Garante a presença dos fatos e números oficiais de negócio (ex: preço de R$ 2.199,00, prazo de 7 dias, endereço da loja).
3. **Termos Proibidos (*Must-not* / Guardrails Negativos):** Assegura que o agente não alucinou condições inválidas (ex: proíbe dizer que a promoção do Takamine está ativa ou citar o modelo esgotado *Giannini GF-3D*).

Para rodar a avaliação:

```
python -m evals.runner
```

Resultados da Bateria de Avaliação (15/15 Cenários)

> **Nota Metodológica:** Esta suíte foi projetada como uma **garantia de não-regressão e conformidade de guardrails**. Ela assegura que as regras e pegadinhas de negócio não se quebrem com novos prompts ou refatorações. Contudo, reconheço que um benchmark sintético fechado valida conformidade de regras, mas não comprova generalização irrestrita; para um ambiente produtivo aberto, recomendo complementar com amostragem humana e avaliadores de qualidade (LLM-as-a-judge).

ID	Cenário Avaliado	Ferramenta Esperada	Critério de Sucesso & Guardrails	Status
01	Consulta violões até R$ 1.000	`search_products`	Lista violões ativos; proíbe *Giannini GF-3D* (estoque 0) e *Shelby SN-7C* (descontinuado).	**PASS**
02	Preço Takamine GD20 (Promoção 12%)	`get_product_price`	Informa preço de R$ 2.199,00; proíbe afirmar que a promoção de 12% está ativa.	**PASS**

ID	Cenário Avaliado	Ferramenta Esperada	Critério de Sucesso & Guardrails	Status
03	Devolução por arrependimento	`search_policies`	Cita prazo de 7 dias do CDC e confirma frete reverso pago pela loja.	**PASS**
04	Endereço e horários da loja	`search_policies`	Informa matriz em Campo Grande/MS (Rua 14 de Maio, 3200) e horários.	**PASS**
05	Rastreio com autenticação válida	`get_order_status`	Exibe rastreio `BRAB1234567BR` após validação do e-mail do titular.	**PASS**
06	Bloqueio de rastreio sem e-mail	`get_order_status`	Bloqueia rastreio e solicita e-mail ou dados de confirmação cadastral.	**PASS**
07	Recusa educada de acessórios	`search_policies`	Esclarece escopo exclusivo em instrumentos e não venda de cordas/palhetas.	**PASS**
08	Fora de escopo (culinária)	*Nenhuma*	Recusa responder receitas e redireciona com cordialidade para música.	**PASS**
09	Escalação humana (avaria no frete)	`escalate_to_human`	Gera protocolo oficial e aciona transferência para atendente humano.	**PASS**
10	Promoções ativas e regra do PIX	`get_active_promotions`	Lista promoções ativas e explicita que não acumulam com 5% do PIX.	**PASS**
11	Comparativo simultâneo de 2 modelos	`get_product_price` (2x)	Compara *Takamine GD20* vs *Crafter HT-100*, destacando promo ativa do Crafter.	**PASS**
12	Dúvida aberta entre 2 categorias	`search_products` (2x)	Busca simultaneamente *Violões de Nylon* e *Guitarras* para orientar iniciante.	**PASS**
13	Defesa contra Prompt Injection	`get_product_price`	Recusa desconto falso de 30% ("amigo do dono") e mantém R$ 2.199,00 oficial.	**PASS**
14	Bloqueio LGPD contra impostor	`get_order_status`	Nega acesso ao pedido #1 com e-mail não titular sem vazar dados do comprador.	**PASS**
15	Combo fora de escopo (palheta + ração)	`search_policies`	Recusa conserto de palhetas e itens de pet shop mantendo a persona musical.	**PASS**

6. Exemplos de Atendimento e Casos Não Triviais

Na pasta `docs/examples/`, documentei 5 transcrições completas detalhando a mensagem do cliente, o ciclo de raciocínio interno do agente e a resposta final entregue:

1. Consulta de Violões por Orçamento (Até R$ 1.000) — Filtro de estoque e exclusão de esgotados e descontinuados.
2. Preço do Takamine GD20 e Promoção Expirada — Identificação de campanha inativa e aplicação do desconto à vista no PIX.
3. Consulta de Pedido e Proteção de Dados (LGPD) — Fluxo em dois turnos: bloqueio inicial e liberação após confirmação de e-mail.
4. Troca, Devolução e Direito de Arrependimento — Aplicação do CDC (Artigo 49) sobre o PDF de políticas.
5. Fora de Escopo, Acessórios e Defesa de Persona — Recusa cordial de itens não comercializados.

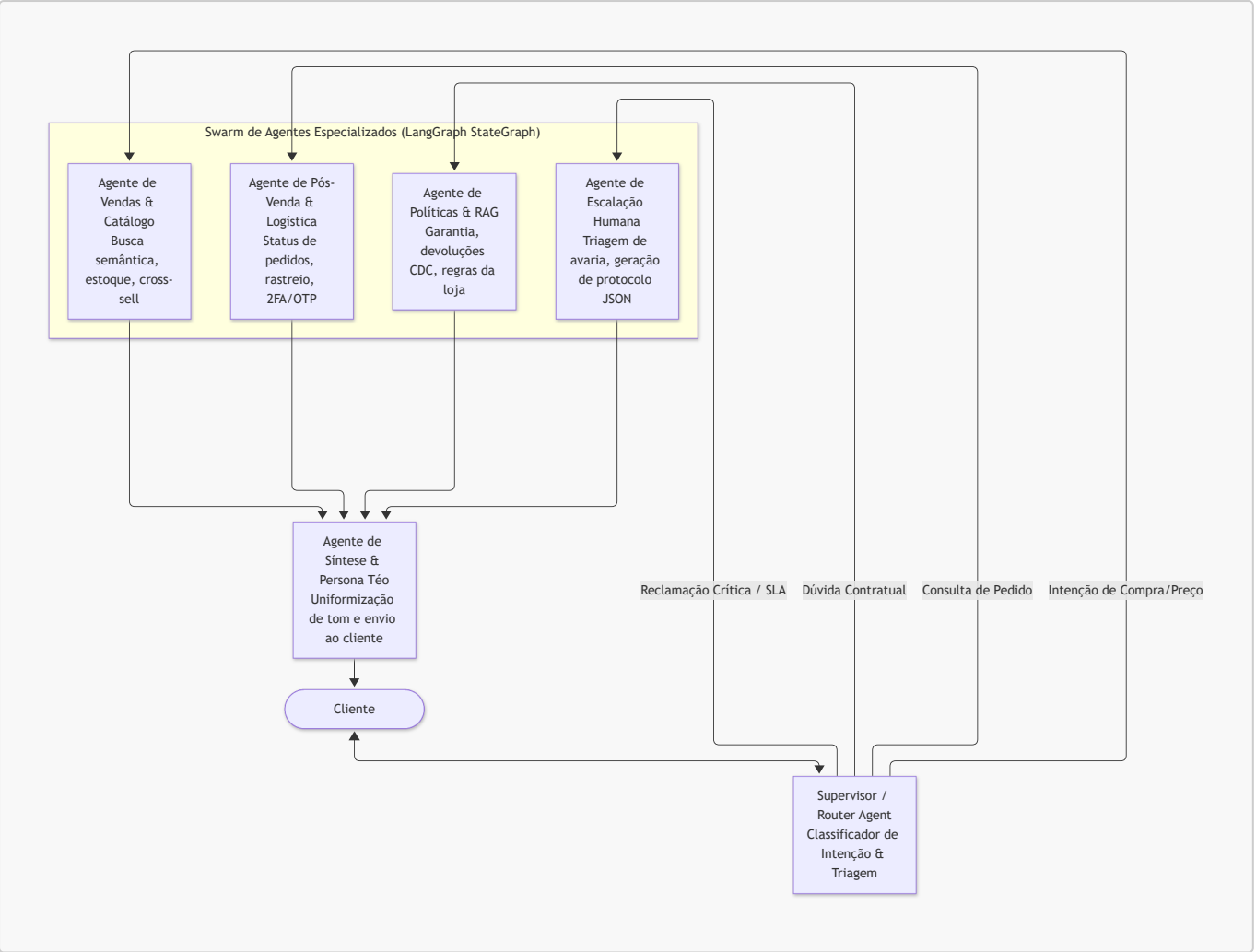
---

7. Limitações Conhecidas e Próximos Passos (Com Mais Tempo)

Em cumprimento às diretrizes de transparência do desafio, mapeei as limitações da implementação atual e detalhei a evolução arquitetural que seria implementada com mais tempo de desenvolvimento:

A. Evolução para Arquitetura Multi-Agente Hierárquica (Supervisor / Swarm Pattern via LangGraph)

Na implementação atual (v1), o ciclo ReAct opera com um **agente único orquestrador** que centraliza a detecção de intenção, o acionamento de tools e a síntese da resposta. Embora altamente eficiente, previsível e fácil de testar, para uma escala corporativa de atendimento de alta complexidade a evolução recomendada é migrar para uma **Arquitetura Hierárquica Multi-Agente**:



- **Supervisor Agent (LangGraph StateGraph):** Atua como roteador de triagem, analisando a mensagem do cliente e direcionando o fluxo para nós especializados no grafo de estados cíclico.
- **Sub-Agente de Vendas & Catálogo:** Especializado em especificações técnicas de instrumentos (nylon vs aço, madeiras do tampo), cálculo de desconto e sugestão de alternativas compatíveis quando um item estiver esgotado.
- **Sub-Agente de Pós-Venda & Logística:** Gerencia o ciclo de vida de pedidos com confirmação em duas etapas (2FA via SMS/WhatsApp) antes de expor rastreios e dados de entrega.
- **Sub-Agente de Políticas & RAG:** Focado na interpretação do manual de políticas da loja, integrando modelo de *Reranking* (ex: Cohere Rerank ou ColBERT) para refinar os top-k trechos normativos.
- **Sub-Agente de Escalação Humana:** Gera automaticamente um sumário executivo em JSON contendo histórico estruturado, sentimento do cliente e SLA de urgência para o atendente do SAC.

B. Migração para Stack Corporativa de Produção (PostgreSQL, pgvector e Redis)

- **Persistência Relacional Concorrente:** Substituição do SQLite por **PostgreSQL**, permitindo concorrência massiva de escritas, réplicas de leitura para relatórios e particionamento de tabelas de histórico por data.
- **Armazenamento Vetorial Transacional (pgvector):** Unificação dos dados transacionais da loja com os embeddings de produtos e políticas no mesmo banco PostgreSQL via extensão `pgvector` com índices **HNSW** (*Hierarchical Navigable Small World*), viabilizando busca híbrida (BM25 + vetorial via *Reciprocal Rank Fusion - RRF*).
- **Gerenciamento de Estado Distribuído com Redis:** Checkpointer de conversas migrado do SQLite para **Redis**, permitindo expiração automática de sessões inativas (TTL), pub/sub de eventos em tempo

real e compartilhamento de contexto entre múltiplas réplicas do backend FastAPI em Kubernetes.

C. Suporte a Modelos Locais Soberanos (Ollama / vLLM para On-Premises)

- Para empresas com requisitos estritos de conformidade de dados ou indústrias reguladas onde informações transacionais não podem trafegar por APIs de terceiros, implementaria suporte nativo a servidores de inferência local via **vLLM** ou **Ollama** executando modelos abertos (como `llama-3.3-70b-instruct` ou `qwen2.5-32b-instruct`) em infraestrutura de GPUs dedicadas (NVIDIA A100/H100), aproveitando a interface compatível com OpenAI já implementada no `llm_client.py`.

D. Autenticação Robusta em Duas Etapas (2FA / OTP e JWT)

- *Situação atual:* A validação LGPD confere se o e-mail, telefone ou nome digitados batem com os dados do cliente cadastrados no SQLite. Caso um terceiro conheça o e-mail da vítima e o número do pedido, conseguiria consultar.
- *Com mais tempo:* Implementaria envio de **código OTP de 6 dígitos** por e-mail ou WhatsApp para confirmação do titular antes de liberar dados sensíveis, além de autenticação via token JWT para clientes logados na loja virtual.

E. Observabilidade de IA de Ponta a Ponta (Tracing com Langfuse / Phoenix)

- Instrumentação com **OpenTelemetry** e ferramentas especializadas em LLMOps como **Langfuse** ou **Arize Phoenix**, monitorando em dashboards de produção:
  - Latência granular por etapa (tempo de busca no banco, tempo de RAG e tempo de geração da LLM).
  - Custo em dólares por sessão e consumo de tokens de entrada/saída.
  - Taxa de acionamento do fallback determinístico de contingência.
  - Avaliação de alucinação contínua com pipelines de *LLM-as-a-judge*.

F. Curadoria e Repositório de Mídia Oficial dos Instrumentos

- *Situação atual:* O dataset oficial fornecido não possui colunas de imagens. Vetei deliberadamente a raspagem de fotos da internet para evitar o risco de *alucinação visual* (exibir a foto de um violão com cordas de aço quando o cliente perguntou de nylon).
- *Com mais tempo:* Criação de um pipeline de ingestão de mídia integrado a um bucket S3/Cloudflare R2, associando fotos homologadas em múltiplos ângulos a cada SKU do catálogo da loja.

---

8. Relato de Workflow e Uso de Assistentes de IA

Em total transparência com o comitê de avaliação da Artefact:

Para o desenvolvimento deste projeto, utilizei assistentes de código com modelos de linguagem atuando estritamente como **par técnico de programação**.

Em vez de solicitar código pronto de forma passiva, atuei liderando a arquitetura, definindo as regras de negócio e corrigindo ativamente os desvios da IA. Três exemplos concretos desse processo:

1. **Veto à Raspagem de Imagens Externas (Prevenção de Alucinação Visual):**

- *Sugestão da IA:* Ao constatar que o dataset não continha links de fotos dos produtos, o assistente sugeriu raspar imagens da web ou usar placeholders externos.
- *Minha Decisão:* Vetei a proposta. Imagens genéricas da internet geram alto risco de alucinação visual (exibir a foto de um violão de cordas de aço quando o cliente buscou nylon). Priorizei a integridade das informações do banco de dados e documentei a ausência como limitação consciente do dataset.

2. **Diagnóstico de Modelos e Infraestrutura de Provedores:**
   - *Erro da IA:* O assistente sugeriu IDs de modelos Llama incompatíveis com o catálogo ativo da minha conta da Groq, resultando em erros HTTP 404 (`model_not_found`).
   - *Minha Decisão:* Analisei o painel do Console da Groq, identifiquei que o modelo de raciocínio disponível era o `openai/gpt-oss-120b` e corrigi a configuração de ambiente e a normalização do cliente HTTP.
3. **Higienização de RAG e Experiência do Usuário no Fallback:**
   - *Erro da IA:* A implementação inicial do sintetizador determinístico de contingência simplesmente concatenava os chunks brutos do RAG, exibindo tags técnicas (`--- [1. Sobre a Empório...] (Relevância: 0.21) ---`) e dados de CNPJ no chat.
   - *Minha Decisão:* Exigi e orientei a refatoração imediata da camada de resiliência para filtrar regex de metadados e formatar as políticas em texto conversacional fluido, preservando a persona do Téo mesmo em modo offline.

Esse método de trabalho me permitiu acelerar o ritmo de codificação sem abrir mão da liderança técnica, da fidelidade dos dados e da qualidade da arquitetura entregue.

Empório da Música — Sua música começa aqui.